MODULE 06 · BLOCK 2 - CONTROL FLOW - CONDITIONAL BLOCKS AND LOOPS

Making decisions: if, elif and else

Which branch runs, why order matters, separate ifs versus one chain, and where an else belongs.

BLOCK 2 of 4 EXAM WEIGHT 29% MODULE 06 of 28

admin@sthithapragna.com

In one if, elif, else chain, **at most one branch runs**: the first whose condition is true. Separate if statements are each checked on their own.

A condition, a colon, an indented block

```
1  temperature = 30
2  if temperature > 25:
3      print("Hot")
4      print("Drink water")
5  print("Done")
```

2 The condition can be any expression. Python tests its truthiness. The colon is required.

3-4 Every line indented under the if runs only when the condition is true.

5 Back at the outer level: this runs either way.

Exactly one of two blocks runs

```
1  age = 15
2  if age >= 18:
3      print("Adult")
4  else:
5      print("Minor")
```

4 else takes no condition, and needs its own colon. It lines up with its if.

2-5 With an else, exactly one of the two blocks always runs.

There is no else if. Python spells it elif. else if x > 3: is a SyntaxError.

Checked top to bottom. The first true branch wins

```
1  score = 85
2  if score >= 90:
3      grade = "A"
4  elif score >= 80:
5      grade = "B"
6  elif score >= 70:
7      grade = "C"
8  else:
9      grade = "F"
10 print(grade)          # B
```

4-5 85 >= 80 is the first true condition. This branch runs, and the chain ends.

6 85 >= 70 is also true, but it is never checked.

8 else is optional. Without it, a chain can run no branch at all.

The widest condition must come last

WRONG ORDER

Everyone gets the first match

```
score = 95
if score >= 70:
    grade = "C"
elif score >= 90:
    grade = "A"
print(grade)
```

Prints C. 95 >= 70 is true first, so the A branch can never run.

RIGHT ORDER

Most demanding test first

```
score = 95
if score >= 90:
    grade = "A"
elif score >= 70:
    grade = "C"
print(grade)
```

Prints A. Ranges tested from the top down.

ON THE EXAM

If an earlier condition includes every value a later one would catch, the later branch is dead code.

elif versus separate if statements

ONE CHAIN

At most one branch

```
x = 5
if x > 0:
    print("a")
elif x > 3:
    print("b")
```

Prints a. The elif is skipped once a branch has run.

TWO DECISIONS

Each if is checked

```
x = 5
if x > 0:
    print("a")
if x > 3:
    print("b")
```

Prints a, then b. Both conditions are true, so both run.

EXAM TECHNIQUE

To count outputs, first mark where each chain starts: every new if starts a new decision.

Any value can be a condition

```
1  name = ""
2  if name:
3      print("Hello", name)
4  else:
5      print("No name given")     # this runs
6  n = 1
7  if n == True:
8      print("equal")             # this runs: 1 == True
```

2 An empty string is falsy, so the else branch runs.

7 True equals 1, so this comparison is true. The same holds for n == 1.

An else belongs to the if it lines up with

ELSE UNDER THE INNER IF

Inner decision has two outcomes

```
1  x = 7
2  if x > 5:
3      if x > 10:
4          print("big")
5      else:
6          print("medium")
```

Prints medium. The else belongs to if x > 10.

ELSE UNDER THE OUTER IF

Inner decision has no else

```
1  x = 7
2  if x > 5:
3      if x > 10:
4          print("big")
5  else:
6      print("small")
```

Prints nothing. x > 5 is true, x > 10 is false, and the inner if has no else.

THE TIEBREAKER

Read indentation, not intention. An else pairs with the if directly above it at the same level.

pass fills a branch that must be empty

```
1  x = 3
2  if x > 0:
3      pass              # nothing to do here yet
4  else:
5      print("not positive")
```

3 pass is a statement that does nothing. It keeps the block legal.

2-5 Here nothing is printed: the if branch runs, and it does nothing.

A comment alone is not enough: a block with only a comment is an IndentationError.

Conditional code that never starts

CODE	ERROR
`if x = 5:`	SyntaxError: Maybe you meant '==' or ':='?
`if x == 5`	SyntaxError: expected ':'
`else if x > 3:`	SyntaxError: use elif
`elif with no if above it`	SyntaxError: invalid syntax
`If x > 3:`	SyntaxError. Python 3.14 adds: Did you mean 'if'?
`code between an if block and its else`	SyntaxError: the else is orphaned

ON THE EXAM

All of these are syntax errors, so the whole file fails before line 1. Nothing prints.

The user types 101

```
1  room = input()
2  if room == 101:
3      print("found")
4  else:
5      print("not found")      # this runs
```

2 room is the str "101". A str never equals an int, so the test is False.

== between a str and an int is quietly False. <, >, <= and >= between them raise TypeError.

Chained comparisons and logic in conditions

```
1  x = 7
2  late = False
3  if 5 <= x <= 10 and not late:
4      print("in range, on time")
5  if x < 0 or x > 5:
6      print("outside 0 to 5")
7  even = x % 2 == 0          # already a bool: False
```

3 Comparisons first, then not, then and. No brackets needed.

7 A comparison produces True or False itself. No if is needed to build a bool.

Two decision tools you may see, but will not be tested on

- **Conditional expression**: size = "big" if x > 2 else "small". One value chosen by a condition.
- **match and case**, from Python 3.10: structural pattern matching. match and case are soft keywords.
- PCEP-30-02 covers if, if-else, if-elif, if-elif-else, and nesting. Answer in those terms.

What to carry into the exam

- One chain: **first true branch wins**, the rest are skipped, even if also true.
- Every new **if** starts a new decision. Separate ifs can all run.
- Put the most demanding condition first, or later branches become dead code.
- An else pairs with the if directly above it at the same indentation.
- No else if, no = in conditions, no empty blocks: all SyntaxErrors.

How many stars does this program print?

```
1  n = 1
2  if n > 0:
3      print("*")
4  if n == True:
5      print("*")
6  elif n:
7      print("*")
8  else:
9      print("*")
```

A One

B Two

C Three

D Four

◆ PAUSE AND DECIDE

How many stars does this program print?

```
1  n = 1
2  if n > 0:
3      print("*")
4  if n == True:
5      print("*")
6  elif n:
7      print("*")
8  else:
9      print("*")
```

Two decisions. First: 1 > 0. Second chain: 1 == True wins, elif and else skipped.

A. One

Treats line 4 as part of the first chain. A new if starts a new decision.

B. Two

One star from each decision. 1 == True, so the second chain stops there.

C. Three

Runs the elif too. Only the first true branch in a chain runs.

D. Four

Treats every branch as independent, else included.

THE SIGNAL

Split the code into chains first, at every if that is not an elif. Then one branch per chain.

What is the output of this program?

```
1  score = 95
2  if score >= 50:
3      grade = "pass"
4  elif score >= 90:
5      grade = "distinction"
6  else:
7      grade = "fail"
8  print(grade)
```

A `pass`

B `distinction`

C `fail`

D pass, then distinction on the next line

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  score = 95
2  if score >= 50:
3      grade = "pass"
4  elif score >= 90:
5      grade = "distinction"
6  else:
7      grade = "fail"
8  print(grade)
```

95 >= 50 is checked first and is true, so the chain ends there.

A `pass`

The first true condition wins. The distinction branch is dead code.

B `distinction`

What the author meant. Python checks in written order, not best match.

C `fail`

else only runs when every condition above it is false.

D pass, then distinction on the next line

Only one branch of a chain runs, and there is only one print.

THE SIGNAL

Python takes the first true branch, not the best one. Check whether an early test swallows a later one.

What is the output of this program?

```
x = 12
if x > 5:
    if x < 10:
        print("A")
else:
    print("B")
print("C")
```

A. B, then C

B. A, then C

C. Only C

D. Nothing is printed

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 12
2  if x > 5:
3      if x < 10:
4          print("A")
5  else:
6      print("B")
7  print("C")
```

12 > 5 is true, so the outer else is skipped. 12 < 10 is false, and the inner if has no else.

A. B, then C
Pairs the else with the inner if. It lines up with the outer one.

B. A, then C
12 < 10 is false, so A is not printed.

C. Only C
Outer test true, inner test false, no inner else. Then C always runs.

D. Nothing is printed
print("C") is outside both ifs, so it always runs.

THE SIGNAL

Before tracing, draw a line from each else up to the if at the same indentation.

The user types 7 and presses Enter.

What happens when this program runs?

```
1  n = input()
2  if n == 7:
3      print("seven")
4  elif n > 5:
5      print("big")
6  else:
7      print("small")
```

A seven is printed

B big is printed

C small is printed

D The code raises a TypeError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  n = input()
2  if n == 7:
3      print("seven")
4  elif n > 5:
5      print("big")
6  else:
7      print("small")
```

n is "7". "7" == 7 is False. "7" > 5 raises TypeError.

A seven is printed

n is the str "7", and a str never equals an int.

B big is printed

Treats n as a number. Comparing it with > is not allowed.

C small is printed

The elif test does not return False. It raises an error.

D The code raises a TypeError

== quietly gives False; > between str and int raises TypeError.

THE SIGNAL

== across types is False. <, >, <= and >= across str and int raise TypeError.

What is the output of this program?

```
1  a, b = 0, "0"
2  if a or not b:
3      print(1)
4  elif not a and b:
5      print(2)
6  else:
7      print(3)
```

A 1

B 2

C 3

D 1 and 2

◆ PAUSE AND DECIDE

What is the output of this program?

```
a, b = 0, "0"
if a or not b:
    print(1)
elif not a and b:
    print(2)
else:
    print(3)
```

a is falsy. b is the non-empty str "0", so it is truthy.

A 1

Treats "0" as falsy. Only the empty string is.

B 2

a or not b is False or False. not a and b is True and "0", which is truthy.

C 3

Would need both conditions false. The elif is truthy.

D 1 and 2

Branches of one chain never both run.

THE SIGNAL

Settle each operand's truthiness first. Then apply not, and, or in that order.

> x has already been set to 5.

Which two lines are syntactically valid? (Choose two.)

A `if x > 3: pass`

B `if x > 3 print(x)`

C `else if x > 3: print(x)`

D `if (x > 3): print(x)`

◆ PAUSE AND DECIDE

Which two lines are syntactically valid? (Choose two.)

A `if x > 3: pass`

A single simple statement may follow the colon on the same line.

B `if x > 3 print(x)`

The colon is missing. SyntaxError: invalid syntax.

C `else if x > 3: print(x)`

There is no else if, and else cannot start a statement alone.

D `if (x > 3): print(x)`

Brackets around a condition are allowed, just unnecessary.

THE SIGNAL

A colon is compulsory; brackets are optional; a one-line body after the colon is legal.